

--	--	--	--	--	--	--	--

Dataset A – Placement Recruitment System

Assessment Objective

This assessment evaluates your ability to build a complete MERN Full Stack application involving:

- Authentication & Authorization
- External API Integration
- Data Validation & Sanitization
- MongoDB Persistence
- Relationship Modeling
- REST API Development
- Querying & Aggregation
- Frontend Integration
- Protected Routes
- State Management
- Deployment Readiness

The assessment focuses on engineering workflows rather than isolated CRUD operations.

Problem Statement

You are required to develop a:

Placement Recruitment Management System

The system manages:

- Students
- Companies
- Placement Drives
- Applications
- Interview Rounds

Students must:

1. Fetch placement dataset from external API
2. Validate and sanitize records
3. Persist valid records into MongoDB Atlas
4. Build secure REST APIs
5. Implement role-based authorization
6. Build React frontend consuming backend APIs
7. Implement analytics & dashboard features

Standard Response Structure

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": []
}
```

Error Response

```
{
  "success": false,
  "message": "Error message"
}
```

Section A — Authentication & Authorization

Objective

Implement JWT-based authentication and role-based access control.

Q1 — Register API

POST /auth/register

Requirements:

- Register new user
- Password hashing mandatory
- Duplicate email prevention
- Store role

Roles:

- admin
- placement_officer
- student

Q2 — Login API

POST /auth/login

Requirements:

- JWT token generation
- Invalid credential handling
- Return authenticated user details

Q3 — Current User API

GET /auth/me

Requirements:

- Protected route

- Return logged-in user details

Authorization Rules

Only:

- admin
- placement_officer

can:

- create drives
- approve applications
- schedule interviews
- access analytics APIs

Students can:

- view own applications
- apply for drives
- view eligible companies

Section B — Dataset Synchronization & MongoDB

Persistence

Objective

Fetch dataset from external API, validate records, sanitize values, and persist valid records into MongoDB Atlas.

Q4 — Sync API

POST /sync

Responsibilities:

- Fetch dataset from private API
- Validate records
- Normalize values
- Reject invalid entries
- Prevent duplicate insertion
- Persist valid records into MongoDB
- Return sync summary

Example Response

```
{
  "success": true,
  "totalFetched": 100,
  "inserted": 82,
  "duplicates": 5,
  "rejected": 13
}
```

Required Sanitization Examples

Raw Value	Expected Value
" arun kumar "	"Arun Kumar"
"ARUN@MAIL.COM"	"arun@mail.com"
"cse"	"CSE"

Validation Requirements

Reject:

- invalid CGPA
- invalid email
- invalid dates
- invalid company references
- duplicate student/application IDs

Q5 — Health API

GET /health

```
Example:
{
  "success": true,
  "database": "connected",
  "documentCount": 82
}
```

Section C — MongoDB Persistence, Relationships & Runtime Validation

Objective

Implement MongoDB persistence with proper relationships, validation logic, and runtime data consistency.

Required Collections & Relationships

Student

Fields:

- studentId
- name
- email
- department
- cgpa
- skills
- graduationYear
- phone
- status

Company

Fields:

- companyId
- name
- role
- package
- eligibleDepartments
- minimumCgpa
- driveDate
- status

Drive

Relationships:

- company → Company

Fields:

- driveId
- title
- mode
- location
- registrationDeadline
- rounds
- status

Application

Relationships:

- student → Student
- drive → Drive

Fields:

- applicationId
- currentRound
- status
- appliedAt

Interview

Relationships:

- application → Application

Fields:

- interviewId
- interviewer
- round
- scheduledAt
- result

Section D — CRUD & Workflow APIs

Objective

Implement REST APIs with proper validation and workflow handling.

Q6 — Student APIs

GET /students
GET /students/:id

Q7 — Company APIs

POST /companies
GET /companies
GET /companies/:id
PATCH /companies/:id
DELETE /companies/:id

Q8 — Drive APIs

POST /drives
GET /drives
GET /drives/:id
PATCH /drives/:id
DELETE /drives/:id

Q9 — Application APIs

POST /applications
GET /applications
GET /applications/:id
PATCH /applications/:id
DELETE /applications/:id

Workflow Rules

- Student CGPA must satisfy company minimum CGPA
- Student department must be eligible
- Duplicate applications not allowed
- Closed drives cannot accept applications

Section E — Interview Workflow APIs

Objective

Implement protected workflow APIs.

Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET A

Q10 — Schedule Interview

POST /interviews

Requirements:

- Only admin/placement_officer allowed
- Application must exist
- Interview date must be valid

Q11 — Update Interview Result

PATCH /interviews/:id

Allowed Results:

- pending
- pass
- fail

Workflow Rules

- Rejected applications cannot receive interviews
- Selected candidates cannot be rescheduled

Section F — Filtering, Search & Pagination

Objective

Implement scalable querying APIs.

Q12 — Student Filters

Examples:

GET /students?department=CSE

GET /students?cgpaMin=8

GET /students?status=active

Q13 — Drive Filters

Examples:

GET /drives?status=open

GET /drives?company=TechNova

Q14 — Pagination & Search

Examples:

GET /applications?page=1&limit=10

GET /applications?search=TechNova

Combined Query Example

GET /applications?

status=shortlisted&

page=2&

limit=5

Section G — Analytics & Aggregation APIs

Objective

Implement MongoDB aggregation APIs.

Q15 — Placement Analytics

GET /analytics/placements

Return:

- total applications
- shortlisted count
- selected count
- rejected count

Q16 — Department Analytics

GET /analytics/departments

Return:

- department-wise placement count
- placement percentage

Q17 — Company Analytics

GET /analytics/companies

Return:

- company-wise selected students
- highest package
- drive participation count

Section H — Frontend Integration

Objective

Build React frontend consuming backend APIs.

Mandatory Frontend Features

Authentication

- Login page
- JWT token handling
- Protected routes
- Role-based rendering

Dashboard

Display:

- analytics cards
- upcoming drives
- shortlisted students
- recent interviews

Student Module

- view eligible companies
- apply for drives
- track application status

Placement Officer Module

- manage drives
- schedule interviews
- update results
- access analytics

Mandatory State Management

Use:

- Context API
- Reducer

Section I — Frontend Evaluator Compliance, Global State & Protected Routes

Objective

Evaluate frontend architecture.

Requirements

Implement:

- Protected routes
- Role-based navigation
- Reducer logic
- Global state management
- Required data-testid attributes
- Dynamic UI updates based on global state

Mandatory Global State

Your application must maintain global state for:

- authenticated user
- JWT token

- students
- companies
- drives
- applications
- filters/search values
- analytics/dashboard data

Use:

- Context API
- Reducer

Mandatory data-testid Attributes

Authentication

Component	data-testid
Login form	login-form
Email input	email-input
Password input	password-input
Login button	login-btn
Logout button	logout-btn

Navigation

Component	data-testid
Navbar	navbar
Dashboard link	dashboard-link
Students link	students-link
Companies link	companies-link
Drives link	drives-link
Applications link	applications-link

Dashboard

Component	data-testid
Analytics container	analytics-container
Total students card	total-students-card
Total companies card	total-companies-card
Total drives card	total-drives-card
Total applications card	total-applications-card
Placement analytics chart	placement-chart
Recent interviews section	recent-interviews

Student Module

Component	data-testid
Student table	student-table
Student search input	student-search
Student filter dropdown	student-filter
Student row	student-row

Drive Module

Component	data-testid
Drive list	drive-list
Create drive button	create-drive-btn
Apply button	apply-btn
Drive search input	drive-search
Pagination next button	pagination-next
Pagination previous button	pagination-prev

Application Module

Component	data-testid
Application table	application-table
Application status filter	application-status-filter
Application row	application-row

Interview Module

Component	data-testid
Interview table	interview-table
Schedule interview button	schedule-interview-btn
Interview result dropdown	interview-result-dropdown

Section J — Frontend Evaluator Compliance & Deployment

Objective

Validate deployment readiness, frontend evaluator compatibility, and runtime UI behavior using automated Playwright-based evaluation.

Mandatory Frontend Behavior

Your frontend must:

- consume your own backend APIs
- update UI dynamically after CRUD operations
- maintain JWT token in global state
- maintain authenticated session after refresh
- restrict unauthorized route access
- render UI based on user role

Mandatory React Router Routes

Students must implement the following routes using React Router DOM.

Route	Purpose
/login	Authentication
/dashboard	Analytics dashboard
/students	Student management
/companies	Company management
/drives	Placement drives
/applications	Application management
/interviews	Interview workflows
/profile	Current user profile

Global State Exposure Requirement

To support automated Playwright evaluation, students must expose global application state using:

window.appState

The global state object must dynamically reflect the current application state.

Mandatory window.appState Structure

```
window.appState = {
  authUser,
  token,
  students,
  companies,
  drives,
  applications,
  interviews,
  filters,
  analytics
}
```